사내 시스템을 MCP 서버로 (실습)

4-1. MCP 서버 만들기 (실습)
4-2. 서버 실행과 검증 (실습)
4-3. 추가로 해볼 만한 것

4-1. MCP 서버 만들기 (실습)

> 교육용 더미 사내 데이터를 fastmcp로 MCP 도구 4개로 노출하는 서버를 만들어봅니다.

fastmcp 라이브러리를 쓰면 MCP 서버 작성이 @tool 도구 정의만큼 간단해집니다. 패턴은 세 줄로 요약됩니다.

1. `mcp = FastMCP("서버이름")` 으로 서버 인스턴스 생성
2. 노출할 함수에 `@mcp.tool()` 데코레이터 (docstring이 곧 도구 설명이라서 1번째 세션 1-1의 원칙이 그대로 적용됩니다)
3. `mcp.run()` 으로 실행 (기본 stdio 모드)

이번 실습에서는 직접 만든 교육용 더미 사내 데이터 3종(임직원, 자산, 프로젝트)을 MCP 도구로 노출합니다.
실제 회사라면 이 자리에 인사 시스템 API와 자산 관리 DB가 들어갑니다.
오늘 만드는 이 서버와 JSON 3종은 6일차 공통 checkpoint에 그대로 들어갑니다. 6일차에서는 Multi-Agent의 data_agent가 이 도구를 씁니다.

백데이터는 `employees.json`, `assets.json`, `projects.json` 세 파일입니다. **파일을 주고받으면 인코딩이 깨지거나 경로가 틀어지기 쉬워서 아래 스크립트로 직접 만들어 씁니다.**

임직원 5명(김하늘, 박도윤, 이서준, 최민준, 정하은), 자산 8건, 프로젝트 4건이 들어갑니다. 모두 교육용 더미 데이터이며 실제 인물이나 조직과 무관합니다. 5, 6, 7일차가 이 데이터를 그대로 이어 쓰므로 값은 바꾸지 않습니다.

따라하기

1. 백데이터를 만듭니다. 아래 코드를 `day4_practice/make_data.py` 로 저장하고 실행하면 같은 폴더에 JSON 세 개가 만들어집니다.

```
# make_data.py - Day4 백데이터 3종을 만드는 스크립트 (교육용 더미 데이터)
"""실행하면 같은 폴더에 employees.json, assets.json, projects.json을 만듭니다.

실행: python make_data.py

5, 6, 7일차가 이 데이터를 그대로 이어 쓰므로 값은 바꾸지 마세요.
"""
import json
import os

BASE = os.path.dirname(os.path.abspath(__file__))

EMPLOYEES = [
    {"id": 1, "name": "김하늘", "team": "클라우드운영팀", "role": "팀장", "email":
"haneul.kim@samsungsds.example.com"},
    {"id": 2, "name": "박도윤", "team": "물류플랫폼팀", "role": "백엔드", "email":
"doyun.park@samsungsds.example.com"},
    {"id": 3, "name": "이서준", "team": "클라우드운영팀", "role": "인프라 엔지니어",
"email": "seojun.lee@samsungsds.example.com"},
    {"id": 4, "name": "최민준", "team": "물류플랫폼팀", "role": "프론트엔드", "emai
l": "minjun.choi@samsungsds.example.com"},
    {"id": 5, "name": "정하은", "team": "클라우드운영팀", "role": "데이터 분석가", "e
mail": "haeun.jung@samsungsds.example.com"},
]

# 노트북 5대, 모니터 2대, GPU 서버 1대로 총 8건입니다. GPU 서버는 6일차 MCP 통합 실습이 조회
하므로 빼지 마세요.
# 5-2의 총액 계산(노트북 5대 x 200만원 = 1,000만원)이 이 구성에 맞춰져 있으니 건수를 바꾸지
마세요.
ASSETS = [
    {"asset_id": "A-1001", "type": "노트북", "model": "갤럭시북4 프로", "owner":
"김하늘", "status": "사용중"},
    {"asset_id": "A-1002", "type": "노트북", "model": "갤럭시북4 프로", "owner":
"박도윤", "status": "사용중"},
    {"asset_id": "A-1003", "type": "노트북", "model": "갤럭시북4", "owner": "이서
준", "status": "사용중"},
    {"asset_id": "A-1004", "type": "노트북", "model": "갤럭시북4", "owner": Non
e, "status": "재고"},
    {"asset_id": "A-1005", "type": "노트북", "model": "갤럭시북4 프로", "owner":
"최민준", "status": "사용중"},
    {"asset_id": "A-1006", "type": "모니터", "model": "뷰피니티 S8", "owner":
"김하늘", "status": "사용중"},
    {"asset_id": "A-1007", "type": "모니터", "model": "뷰피니티 S8", "owner":
"정하은", "status": "사용중"},
    {"asset_id": "A-1008", "type": "GPU 서버", "model": "NVIDIA A100 80GB",
"owner": "이서준", "status": "사용중"},
]
```

```
PROJECTS = [
    {"project_id": "P-2601", "name": "차세대 물류 플랫폼 구축", "team": "물류플랫폼
팀", "status": "진행중", "member_count": 12},
    {"project_id": "P-2602", "name": "클라우드 비용 최적화", "team": "클라우드운영
팀", "status": "진행중", "member_count": 7},
    {"project_id": "P-2603", "name": "물류 관제 대시보드 고도화", "team": "물류플랫폼
팀", "status": "진행중", "member_count": 5},
    {"project_id": "P-2604", "name": "사내 AI 어시스턴트 도입", "team": "클라우드운
영팀", "status": "진행중", "member_count": 4},
]


def write_json(filename: str, rows: list) -> str:
    """한글이 깨지지 않게 UTF-8과 ensure_ascii=False로 씁니다."""
    path = os.path.join(BASE, filename)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(rows, f, ensure_ascii=False, indent=2)
    return path


if __name__ == "__main__":
    for filename, rows in [("employees.json", EMPLOYEES),
                           ("assets.json", ASSETS),
                           ("projects.json", PROJECTS)]:
        write_json(filename, rows)
        print(f"생성: {filename} ({len(rows)}건)")

    laptops = [a for a in ASSETS if a["type"] == "노트북"]
    total = len(laptops) * 2000000
    print(f"확인: 자산 {len(ASSETS)}건 중 노트북 {len(laptops)}대, 한 대당 200만원이면
총액 {total:,}원")
```

```
python make_data.py
```

생성된 파일명과 건수, 그리고 노트북 5대 기준 총액이 출력되면 정상입니다.

2. 서버 파일을 만듭니다. 아래 코드를 `day4_practice/mcp_server.py` 로 저장합니다. 데이터는 코드에 넣지 않고 1번에서 만든 JSON에서 로드합니다.

```
# mcp_server.py - 삼성SDS 교육용 더미 사내 데이터 MCP 서버 (fastmcp)
"""노출하는 도구 4개:
  - search_employee(name)       임직원 개인 조회
  - list_team_members(team)     팀별 임직원 목록
  - get_asset(owner)            자산 조회
  - list_projects(team)         프로젝트 목록

실행: python mcp_server.py
(stdio 모드로 대기합니다. 단독으로 쓰는 파일이 아니라
 클라이언트가 자식 프로세스로 띄워서 사용합니다)
"""
import json
import os
from fastmcp import FastMCP

mcp = FastMCP("sds-company-data")

# make_data.py가 만든 JSON 3개를 코드와 같은 폴더에서 로드합니다 (교육용 더미 데이터)
BASE = os.path.dirname(os.path.abspath(__file__))
EMPLOYEES = json.load(open(os.path.join(BASE, "employees.json"), encoding="u
tf-8"))
ASSETS = json.load(open(os.path.join(BASE, "assets.json"), encoding="utf-
8"))
PROJECTS = json.load(open(os.path.join(BASE, "projects.json"), encoding="utf
-8"))


@mcp.tool()
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다.

    직원의 소속 팀, 직무, 이메일이 필요할 때 사용한다.
    팀 전체 명단이나 인원수에는 사용하지 않는다 (list_team_members 사용).
    자산이나 프로젝트 질문에도 사용하지 않는다 (get_asset, list_projects 사용).

    Args:
        name: 직원의 정확한 한국어 이름
    """
    for emp in EMPLOYEES:
        if emp["name"] == name:
            return json.dumps(emp, ensure_ascii=False)
    names = ", ".join(e["name"] for e in EMPLOYEES)
    return f"'{name}' 직원을 찾을 수 없습니다. 등록된 직원: {names}"


@mcp.tool()
def list_team_members(team: str) -> str:
```

```
    """팀 이름으로 소속 임직원 목록을 조회한다.

    특정 팀의 인원수나 명단이 필요할 때 사용한다.
    개별 직원의 상세 정보는 search_employee를 사용한다.

    Args:
        team: 팀 이름 (예: '클라우드운영팀', '물류플랫폼팀')
    """
    rows = [e for e in EMPLOYEES if e["team"] == team]
    if not rows:
        teams = sorted({e["team"] for e in EMPLOYEES})
        return f"'{team}' 팀을 찾을 수 없습니다. 등록된 팀: {teams}"
    return json.dumps({"count": len(rows), "members": rows}, ensure_ascii=Fa
lse, indent=2)


@mcp.tool()
def get_asset(owner: str = "") -> str:
    """사내 자산(노트북, 모니터 등)을 조회한다.

    Args:
        owner: 사용자 이름으로 필터. 빈 문자열이면 전체 자산 반환
    """
    rows = [a for a in ASSETS if not owner or a["owner"] == owner]
    if not rows:
        return f"'{owner}' 사용자의 자산이 없습니다."
    return json.dumps({"count": len(rows), "assets": rows}, ensure_ascii=Fal
se, indent=2)


@mcp.tool()
def list_projects(team: str = "") -> str:
    """진행 중인 프로젝트 목록을 조회한다.

    Args:
        team: 팀 이름으로 필터 (예: '물류플랫폼팀'). 빈 문자열이면 전체
    """
    rows = [p for p in PROJECTS if not team or p["team"] == team]
    if not rows:
        return f"'{team}' 팀의 프로젝트가 없습니다."
    return json.dumps(rows, ensure_ascii=False, indent=2)


if __name__ == "__main__":
    mcp.run()   # stdio 모드로 실행
```

3. 만들어진 JSON 3개가 `mcp_server.py`와 같은 폴더에 있는지 확인한 뒤 한 번 실행해 봅니다. 아무것도 출력하지 않고 멈춘 것처럼 보이면 정상입니다. stdio 모드는 표준 입력으로 들어올 요청을 기다리는 상태이며 Ctrl+C로 종료합니다.

```
python mcp_server.py
```

MCP는 텍스트 기반 프로토콜이므로 반환값은 문자열로 통일합니다. 구조화된 데이터는 `json.dumps(ensure_ascii=False)`로 직렬화(구조를 가진 데이터를 문자열 한 줄로 바꾸는 일)하는 것이 관례입니다.

막힐 때

- `FileNotFoundError: employees.json`이 납니다.
  `make_data.py`를 아직 실행하지 않았거나 다른 폴더에서 실행한 경우입니다. `mcp_server.py`와 같은 폴더에서 `python make_data.py`를 먼저 돌리고 JSON 세 개가 생겼는지 확인합니다.
- **stdio 서버 코드 안에서 print()를 쓰면 안 됩니다.**

  표준 출력이 곧 프로토콜 통신 채널이라서 print가 섞이면 클라이언트가 JSON 파싱에 실패해 알 수 없는 연결 에러가 납니다. 디버깅 출력이 필요하면 `import sys` 후 `print(..., file=sys.stderr)`처럼 표준 에러로 보냅니다. MCP 실습에서 가장 흔한 사고 원인입니다.
- `@mcp.tool()`은 괄호를 붙여 씁니다.

  괄호 없는 표기가 되는 버전도 있지만 fastmcp 버전에 따라 다르므로 괄호로 통일합니다.

확인: 실행했을 때 에러 없이 대기 상태로 들어가고 Ctrl+C로 종료되면 성공입니다. 도구가 실제로 동작하는지는 다음 절에서 검증합니다.

> fastmcp 데코레이터로 사내 데이터 조회 함수 4개를 표준 MCP 도구로 노출했습니다.

4-2. 서버 실행과 검증 (실습)

> 파이썬 MCP 클라이언트 스크립트로 서버의 도구 목록과 호출 결과를 검증해봅니다.

서버를 만들었으면 Agent에 붙이기 전에 서버 단독으로 검증합니다. 검증 항목은 두 가지입니다.

1. 도구 목록이 의도대로 노출되는가 (list_tools)
2. 각 도구가 올바른 결과를 반환하는가 (call_tool)

검증은 mcp 파이썬 SDK의 ClientSession으로 클라이언트를 직접 짜서 진행합니다. 이 과정에서 MCP의 요청과 응답 구조도 눈으로 확인하게 됩니다.

공식 웹 UI 검증 도구인 MCP Inspector도 있습니다. 다만 Node.js가 필요합니다. 그래서 주 경로는 파이썬 클라이언트 스크립트로 두고 Node.js가 설치된 환경에서만 Inspector로도 확인해 봅니다.

여기서 async(비동기)가 처음 등장하므로 최소한만 짚고 갑니다. MCP 통신은 프로세스 사이의 입출력 대기가 많아 파이썬 SDK가 비동기 방식으로 만들어져 있습니다. 규칙은 세 가지만 기억하면 됩니다.

- `async def` 로 정의한 함수는 코루틴입니다. 그냥 호출하면 실행되지 않고 예약표만 돌아옵니다.
- 코루틴 안에서 다른 코루틴을 실행할 때는 앞에 `await` 를 붙입니다. 결과가 올 때까지 이 지점에서 기다리되 그동안 다른 일을 해도 된다는 뜻입니다.
- 프로그램 진입점에서는 `asyncio.run(main())` 으로 최상위 코루틴을 실행합니다.

Java로 비유하면 CompletableFuture 체인을 언어 문법으로 만든 것이 async/await입니다. 이 실습에서는 SDK가 async를 요구하니 규칙대로 감싼다 수준이면 충분합니다.

따라하기

1. 검증 스크립트를 만듭니다. 아래 코드를 `day4_practice/mcp_verify.py` 로 저장합니다.

```
# mcp_verify.py — 파이썬 MCP 클라이언트로 서버를 직접 검증
import asyncio
import os
import sys
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

SERVER_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "mcp_
server.py")


async def main():
    # 1) 서버를 자식 프로세스로 띄울 방법을 기술
    #    sys.executable: 현재 파이썬 인터프리터 경로 (python/python3 문제 회피)
    params = StdioServerParameters(command=sys.executable, args=[SERVER_PAT
H])

    # 2) stdio로 연결하고 세션 시작
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()   # MCP 초기 핸드셰이크

            # 3) 검증 1: 도구 목록 확인
            tools = await session.list_tools()
            print("=== 노출된 도구 목록 ===")
            for t in tools.tools:
                print(f"- {t.name}: {t.description.splitlines()[0]}")

            # 4) 검증 2: 도구 호출
            print("\n=== 도구 호출 검증 ===")
            r1 = await session.call_tool("search_employee", {"name": "김하
늘"})
            print(f"search_employee -> {r1.content[0].text}")

            r2 = await session.call_tool("list_team_members", {"team": "클라우
드운영팀"})
            print(f"list_team_members -> {r2.content[0].text[:80]}")

            r3 = await session.call_tool("get_asset", {"owner": "김하늘"})
            print(f"get_asset -> {r3.content[0].text}")

            r4 = await session.call_tool("list_projects", {"team": "물류플랫폼
팀"})
            print(f"list_projects -> {r4.content[0].text}")

            # 5) 검증 3: 에러 케이스도 확인
            r5 = await session.call_tool("search_employee", {"name": "홍길
```

```
동"})
            print(f"없는 직원 -> {r5.content[0].text}")


if __name__ == "__main__":
    asyncio.run(main())
```

2. 실행하면 도구 4개 목록과 호출 결과 5건이 출력됩니다.

```
=== 노출된 도구 목록 ===
- search_employee: 삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다.
- list_team_members: 팀 이름으로 소속 임직원 목록을 조회한다.
- get_asset: 사내 자산(노트북, 모니터 등)을 조회한다.
- list_projects: 진행 중인 프로젝트 목록을 조회한다.

=== 도구 호출 검증 ===
search_employee -> {"id": 1, "name": "김하늘", "team": "클라우드운영팀", "role":
"팀장", "email": "haneul.kim@samsungsds.example.com"}
list_team_members -> {"count": 3, "members": [{"id": 1, "name": "김하늘", ...
get_asset -> {"count": 2, "assets": [...]}
list_projects -> [{"project_id": "P-2601", ...}]
없는 직원 -> '홍길동' 직원을 찾을 수 없습니다. 등록된 직원: 김하늘, 박도윤, 이서준, 최민준, 정하
은
```

3. (선택) Node.js가 설치된 환경이라면 MCP Inspector로도 같은 검증을 웹 UI에서 해 봅니다. 브라우저가 열리면 Connect 후 Tools 탭에서 목록 확인과 호출 테스트가 가능합니다.

```
npx @modelcontextprotocol/inspector python mcp_server.py
```

막힐 때

- 연결이 즉시 종료되거나 `FileNotFoundError` 가 납니다.

  SERVER_PATH가 틀렸거나 4-1의 `make_data.py` 를 실행하지 않아 JSON이 없는 경우가 대부분입니다. 이 파일은 클라이언트 쪽이라 print를 써도 되므로 SERVER_PATH를 print로 찍어 파일이 실제로 있는지 먼저 확인합니다.

- 서버 쪽에서 `ImportError: fastmcp` 가 납니다.

  클라이언트와 서버의 파이썬 환경이 다른 경우(가상환경 문제)입니다. `command=sys.executable` 을 쓰면 같은 인터프리터로 통일되어 예방됩니다.

- 연결 에러 메시지가 unexpected token 류입니다.

  4-1에서 말한 서버 코드 안의 print가 원인일 확률이 높습니다.

확인: 도구 목록에 search_employee, list_team_members, get_asset, list_projects 4개가 보이고 다섯 건의 호출이 모두 의도한 결과를 반환하면 성공입니다.

> Agent에 붙이기 전에 MCP 서버를 클라이언트 스크립트로 단독 검증하는 절차를 익혔습니다.

4-3. 추가로 해볼 만한 것

▼ 리소스와 프롬프트 노출해 보기

- MCP 서버에 도구 외의 나머지 두 가지, 리소스와 프롬프트도 노출해 봅니다. `mcp_server.py` 에 아래를 추가하고 `mcp_verify.py` 에서 `await session.list_resources()` 와 `await session.list_prompts()` 로 확인합니다. 실무 비중은 Tools가 압도적이므로 개념 확인 수준이면 충분합니다.

```
# mcp_server.py 에 추가

# Resource: URI 기반 데이터. 도구는 "호출"이고 리소스는 "읽기"
@mcp.resource("employees://{employee_id}")
def get_employee_resource(employee_id: str) -> str:
    """직원 상세 정보를 URI로 제공한다. 예: employees://1"""
    emp = next((e for e in EMPLOYEES if e["id"] == int(employee_id)), {})
    return json.dumps(emp, ensure_ascii=False)


# Prompt: 미리 정의된 프롬프트 템플릿. 클라이언트가 명령 형태로 노출 가능
@mcp.prompt()
def asset_report(team: str) -> str:
    """팀별 자산 현황 보고서 작성용 프롬프트."""
    return f"""{team}의 자산 현황을 조사해 보고서를 작성하세요.
1. search_employee로 팀원을 확인하고
2. get_asset으로 팀원별 자산을 조회하고
3. 유형별 수량을 표로 정리하고 재고 자산이 있으면 배정 제안을 덧붙이세요."""
```

- MultiServerMCPClient 설정에 두 번째 서버 항목을 추가해 서버 여러 개를 동시에 연결하는 구조를 확인해 봅니다. 4-1 서버를 복사해 이름만 바꿔도 됩니다.
- 5번째 세션 5-2의 질문 세트에 도구가 필요 없는 질문("안녕하세요라고만 인사해 주세요")을 추가해 도구 미사용 경로도 통과하는지 확인해 봅니다.